

Network Simulation CN

1.Project Overview

1.1 Introduction:

Welcome to our powerful network simulation tool. Explore the intricacies of network architecture, test configurations, and analyze performance - all from the comfort of your browser.

1.2 Objective:

- Simulate different layer functions.
- Enable testing of static and dynamic routing protocols (RIP/OSPF).
- Provide visualization and logs for network activities.
- Facilitate user-defined network topologies and configuration.

commits:

[src/main/java/com/networksimulator](#)

[src/main/java/com/networksimulator](#)

[target](#)

[target](#)

[Dummy_IP_Table.csv](#)

[Dummy_IP_Table.csv](#)

[IP_Learning_Table.csv](#)

[IP_Learning_Table.csv](#)

[MAC_Learning_Table.csv](#)

[MAC_Learning_Table.csv](#)

[README.md](#)

[README.md](#)

[README_NetworkSimulator.md](#)

[README_NetworkSimulator.md](#)

[pom.xml](#)

[pom.xml](#)

Network Simulator

Permalink: Network Simulator

A Java-based network simulation tool that visualizes and simulates network topologies using DOT files and Graphviz.

2.System Architecture:

2.1 High Level Design

- **User Interface:** CLI/GUI for network design and simulation control.
- **Core Simulation Engine:** Manages device objects, link creation, and event scheduling.
- **Network Layer Module:** Handles IP addressing (classless/CIDR), ARP, and routing logic.
- **Routing Protocols:** Implements static and dynamic protocols (RIP/OSPF).
- **Packet Forwarding Engine:** Simulates packet transmission and routing table lookups.
- **Logger/Visualizer:** Provides logs and visualization of simulation state.

2.2 Component Diagram

- Devices: Routers, Hosts, Switches, Hubs, Bridge , EndDevices
- Links: Point-to-point, Broadcast
- ARP Table, Routing Table, Interface Configurations , Error Correction , Control Protocols



Project Structure

Permalink:  **Project Structure**

```
network-simulator/
├── src/ # Java source files
├── target/ # Maven build output
├── topology.dot # Graphviz DOT file defining the network topology
├── topology.png # Rendered image of the topology
├── pom.xml # Maven configuration file
├── .vscode/ # VSCode settings (optional)
└── .git/ # Git repository metadata
```

3.Features and Functionalities

3.1 Device Creation

- Add routers, switches, hosts with unique IDs.
- Configure network interfaces and assign IP addresses/subnets.

3.2 Topology Configuration

- Support for custom network topologies.
- Users can connect devices via links, define network segments.

3.3 IP Addressing

- Implements classless IPv4 addressing (CIDR).
- Supports subnetting, supernetting, and address allocation.

3.4 Address Resolution Protocol (ARP)

- Simulates ARP requests and responses.
- Handles ARP cache/table management per device.

3.5 Routing

- **Static Routing:** Users can manually add/remove routes.
- **Dynamic Routing:** Implements RIP or OSPF for automatic route computation and table updates.
- Supports longest prefix match for route selection.

3.6 Packet Forwarding

- Simulates packet creation, encapsulation, and forwarding.
- Routing decisions are based on current routing tables.

Requirements

Permalink:  [Requirements](#)

Software Prerequisites

- Java 17+
- Maven
- Graphviz (for generating PNGs from .dot files)

Core Simulation Capabilities

- Routing tables in the simulator are implemented using **longest mask (prefix) matching**. When forwarding a packet, the router examines all route entries and selects the route with the longest subnet mask (i.e., the most specific match) that fits the destination IP. This closely emulates real-world routing table behavior in IP routers.
- The simulator's routing lookup function iterates over all routes, identifying the entry with the longest subnet mask that matches the destination IP address. This ensures that the most specific route is always selected for forwarding, preventing ambiguous or suboptimal routing decisions.

Dijkstra's Shortest Path Algorithm

- The simulator supports running or interacting with application-layer protocols, such as Telnet, FTP, and SSH, to demonstrate end-to-end network communication.
These applications may be developed as independent applets/programs, or pre-existing client programs can be integrated. When network communication is triggered in the simulator, the relevant application is invoked to handle the session.
For example, when a user initiates a Telnet session, the simulator either calls a built-in Telnet client or interfaces with an external Telnet client application.
- The network simulator is designed to support application-level communication by either implementing lightweight versions of protocols (e.g., a simple Telnet/FTP client) or leveraging pre-existing client applications provided by the programming environment.
When a communication session is initiated (such as Telnet, FTP, or SSH), the simulator calls the appropriate external program or module, passing the necessary parameters for the session.
Any pre-existing application or open-source library used for this purpose is fully cited in the References section.



Dependencies

Permalink:  **Dependencies**

From pom.xml:

- junit-jupiter-api (for unit testing)
- graphviz-java (for Graphviz rendering)



Getting Started

Permalink:  [Getting Started](#)

1 Clone the repository

```
git clone  
cd network-simulator
```

2 Build the project

```
mvn clean install
```

3 Run the application

```
mvn exec:java -Dexec.mainClass="com.network.Main"
```

Replace `com.network.Main` with your actual main class if different.

4 View Topology

- Edit the `topology.dot` file to define network nodes and edges.
- Use Graphviz to render it:

```
dot -Tpng topology.dot -o topology.png
```



Running Tests

Permalink: [🧪 Running Tests](#)

```
mvn test
```



Contributing

Permalink: [✍️ Contributing](#)

1. Fork this repository
2. Create your feature branch: `git checkout -b feature/AmazingFeature`
3. Commit your changes: `git commit -m 'Add some AmazingFeature'`
4. Push to the branch: `git push origin feature/AmazingFeature`
5. Open a Pull Request